

Presented by Group 21

Support Vector Machine for Cost-effective Feature Selection

WONG Man Chun, Matthew

TANG Cheuk Him, Thomas

XIONG Zelin

TIN Yiu Tsun, Jason

WEN Meng Qian, Velma

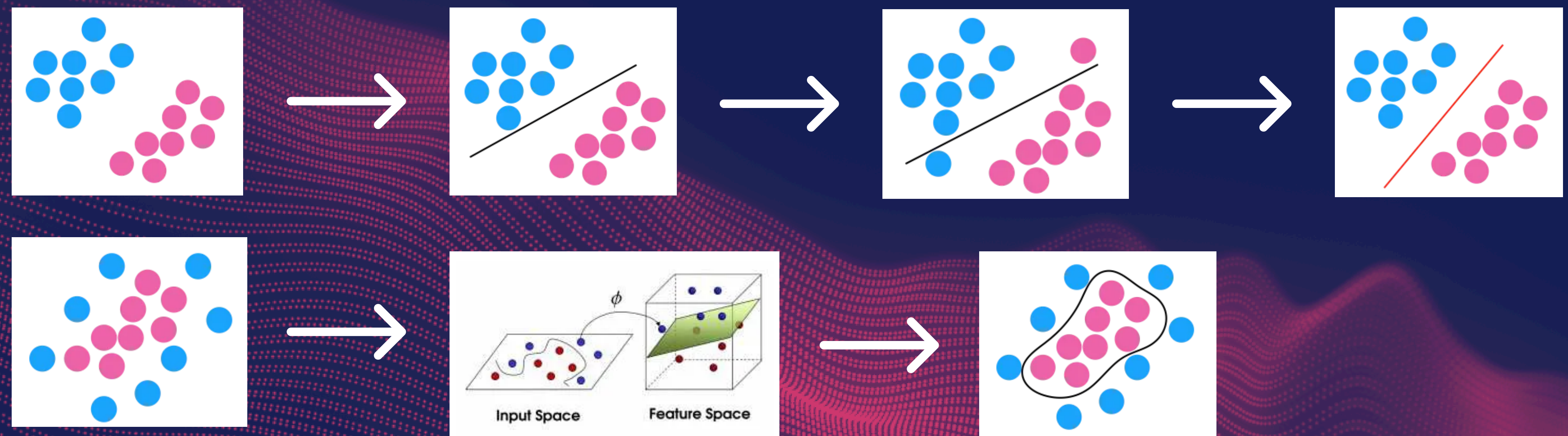
INTRODUCTION

SVM

Support Vector Machine

➤ A supervised learning algorithm for classification and regression tasks.

➤ Find the optimal hyperplane that maximizes the margin between different classes while minimizing classification error



SVM

Support Vector Machine

Maximize the margin between the nearest points of different classes, known as support vectors

GOAL

Maximize this margin while ensuring that no data points fall within our decision boundary as $w \cdot x + b = 0$

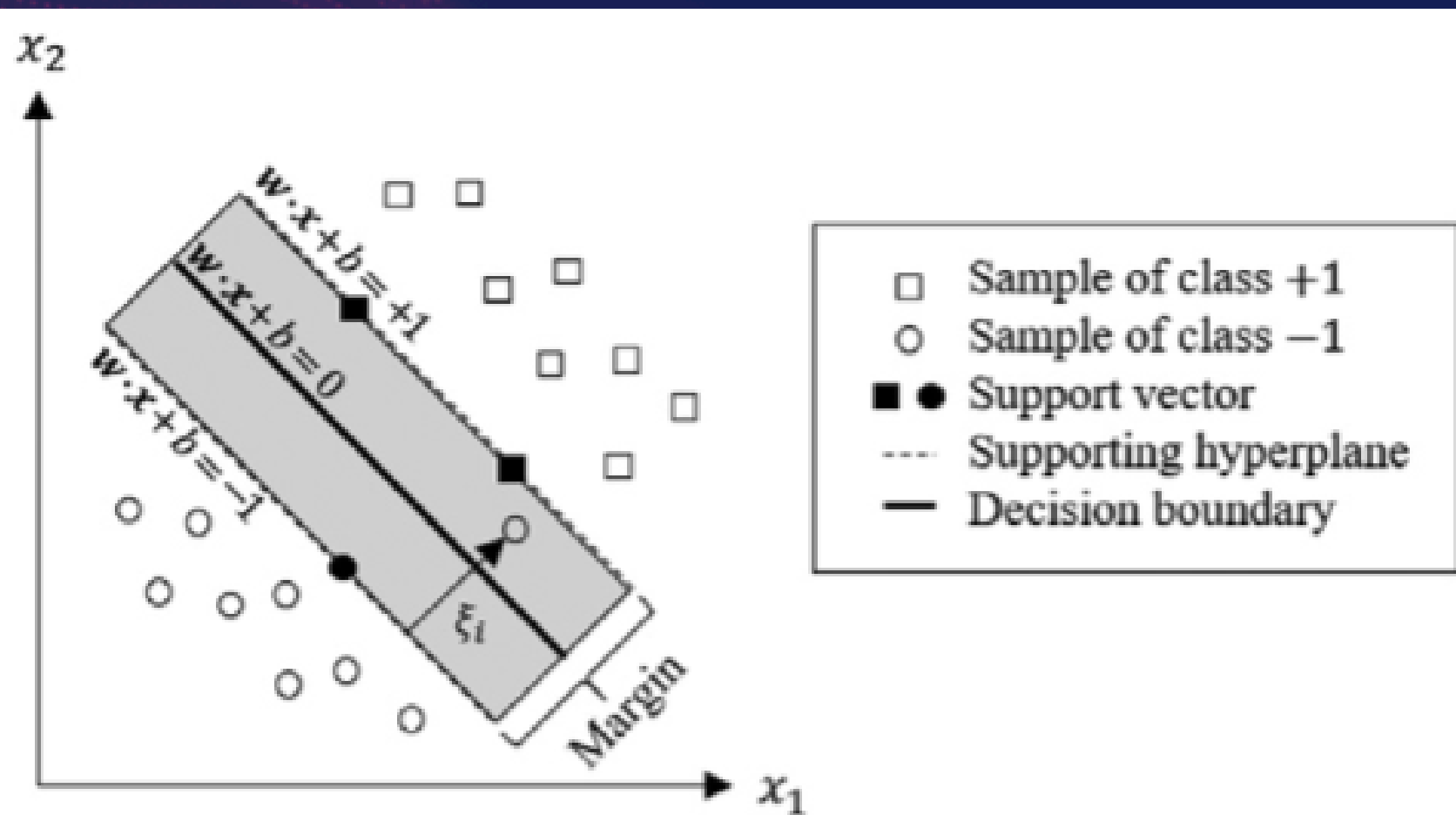
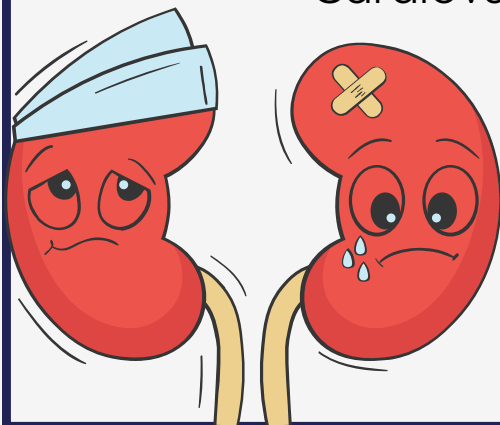


Fig. 1. Illustration of support vector machine.

Introduction TO OUR CASE

Chronic Kidney Disease

CKD affects the kidneys' ability to filter waste and excess fluids from the blood. For example, Fluid Retention, Electrolyte Imbalance, Anemia, Bone Health and Cardiovascular Health.



Importance of Early Detection

Early detection of Chronic Kidney Disease (CKD) is crucial as it enables timely interventions that can **slow disease progression, prevent complications, and improve patients' quality of life**. Additionally, it helps **reduce healthcare costs** by minimizing the need for advanced treatments and hospitalizations associated with end-stage renal disease.

Traditional ML Model Detection

Including concerns like **Overfitting, Model Complexity** and **Generalization**, traditional machine learning models may **not be flexible enough** to handle the complex data

Use L1-SVM for CKD Detection

How can we build a MILP model for L1-SVM and extend it to seek potential CKD patients with a higher accuracy?

How can we build a robust model to address the uncertainty of feature costs, and remove the redundant feature?

What is the comparison in our model with other models (L1-SVM vs GFS-CESVM1 and GFS-RCESVM1) when detecting a CKD patient?

OBJECTIVE

With L1-SVM Model

01

Increase Accuracy

Categorize patients into CKD and non-CKD groups with a high level of accuracy compared to traditional ML model.

02

Identify and Remove

Seek out the most relevant clinical features.
Remove the irrelevant or useless features.

03

Reduce Cost for Patients

Reducing excessive costs and resource limitations due to High-performing features that may lead to impractical solutions, allowing patients to detect CKD at lower cost.

04

Compare and Conclude

Compare L1-SVM vs GFS-CESVM1 vs GFS-RCESVM1 of its accuracy, and effectiveness and determine which is the best.



Presented by Group 21

Support Vector Machine for Cost-effective Feature Selection

WONG Man Chun, Matthew

TANG Cheuk Him, Thomas

XIONG Zelin

TIN Yiu Tsun, Jason

WEN Meng Qian, Velma

MODEL SETUP

Problem of L1-SVM

- **Ignoring cost consideration**

Making insensible and ineffective decisions

To solve the problem:

Extend SVM by adding budget constraints (B)

Perfect solution: CESVM !

Cost-effective l1-SVM (CESVM1)

$$\text{Minimize : } \sum_j z_j + C_2 \cdot \sum_j s_j \cdot \text{cost}_j + C_1 \cdot \sum_i \xi_i$$

$$\underset{w^+, w^-, v, \xi, b}{\text{minimize}} \quad \sum_{j=1}^l (w_j^+ + w_j^-) + C \sum_{i=1}^m \xi_i \quad (3.11)$$

$$\text{s.t.} \quad y_i \left(\sum_{j=1}^l ((w_j^+ - w_j^-)^T \cdot x_{ij}) + b \right) \geq 1 - \xi_i, \quad \forall i, \quad (3.12)$$

$$\sum_{j=1}^l c_j v_j \leq B, \quad (3.13)$$

$$w_j^+ + w_j^- \leq M_j v_j, \quad \forall j, \quad (3.14)$$

$$\xi_i \geq 0, \quad \forall i, \quad (3.15)$$

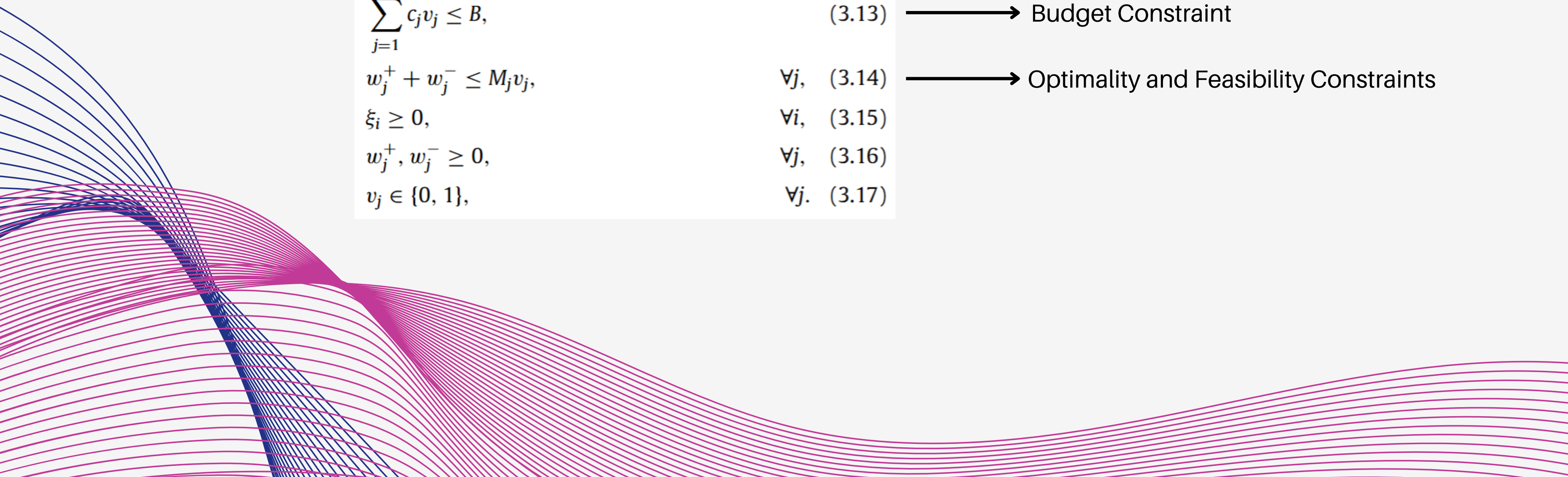
$$w_j^+, w_j^- \geq 0, \quad \forall j, \quad (3.16)$$

$$v_j \in \{0, 1\}, \quad \forall j. \quad (3.17)$$

} L1-SVM

→ Budget Constraint

→ Optimality and Feasibility Constraints



Introducing RCESVM

- **Relies on worst-case analysis**
All possible solutions remain feasible
- **LP model:**
 - over-conservatism by abandoning a considerable degree of optimality for the nominal problem.

RESVM: leveraging the flexibility and computational efficiency of the LP model

Robust and cost-effective l1-SVM (RCESVM1)

CESVM1

$$\sum_{j=1}^l c_j v_j \leq B,$$



RCESVM1

$$\sum_{j=1}^l c_j v_j + \max_{\{S: S \subseteq J, |S| \leq r\}} \left\{ \sum_{j \in S} \hat{c}_j v_j \right\} \leq B$$

Robust and cost-effective l1-SVM (RCESVM1)

$$\beta(\mathbf{v}^*, \Gamma) = \max_{\{S: S \subseteq J, |S| \leq \Gamma\}} \left\{ \sum_{j \in S} \hat{c}_j v_j^* \right\}. \quad (3.19)$$

$\beta(\mathbf{v}^*, \Gamma)$ is equivalent to the objective function of the following linear optimization problem.

$$\beta(\mathbf{v}^*, \Gamma) = \text{maximize} \quad \sum_{j \in J} \hat{c}_j v_j^* z_j \quad (3.20)$$

$$\text{s.t.} \quad \sum_{j \in J} z_j \leq \Gamma, \quad (3.21)$$

$$0 \leq z_j \leq 1, \quad \forall j \in J. \quad (3.22)$$

$$\text{minimize} \quad \sum_{j \in J} p_j + z \Gamma \quad (3.23)$$

$$\text{s.t.} \quad z + p_j \geq \hat{c}_j v_j^*, \quad \forall j \in J, \quad (3.24)$$

$$p_j \geq 0, \quad \forall j \in J, \quad (3.25)$$

$$z \geq 0. \quad (3.26)$$

Presented by Group 21

Support Vector Machine for Cost-effective Feature Selection

WONG Man Chun, Matthew

TANG Cheuk Him, Thomas

XIONG Zelin

TIN Yiu Tsun, Jason

WEN Meng Qian, Velma

CODING

model of l1-SVM

```
model = Model("L1_SVM")
n_features = X_train.shape[1]
w_plus = model.addVars(n_features, lb=0, name="w_plus")
w_minus = model.addVars(n_features, lb=0, name="w_minus")
xi = model.addVars(len(y_train), lb=0, name="xi")
b = model.addVar(lb = 0, name = "b")
C = 100

model.setObjective(quicksum(w_plus[j] + w_minus[j] for j in range(n_features)) + C * quicksum(xi[i] for i in range(len(y_train))), GRB.MINIMIZE)

for i in range(len(y_train)):
    dot_product = quicksum((w_plus[j] - w_minus[j]) * X_train.iloc[i, j] for j in range(n_features))
    model.addConstr(y_train.iloc[i] * (dot_product + b) >= 1 - xi[i])

model.optimize()

if model.status == GRB.OPTIMAL:
    print("Optimal Weights:")
    for j in range(n_features):
        print(f"Feature {j}: w+ = {w_plus[j].X}, w- = {w_minus[j].X}, b : {b.X}")

l1weights_data = {
    'w+': [w_plus[j].X for j in range(n_features)],
    'w-': [w_minus[j].X for j in range(n_features)],
}
l1SVM_b = b.X

l1weights_df = pd.DataFrame(l1weights_data)
print(l1weights_df)
```

data clean and data preparing

```
# 数据加载
file_path = 'data.xlsx'
data = pd.read_excel(file_path)

# 数据清洗
# 去除多余空格或制表符
data.columns = data.columns.str.replace(r'\t', '', regex=True).str.strip()

# 替换 '?' 为 NaN
data.replace('?', pd.NA, inplace=True)

# 去除目标列中的隐藏空格或异常值
if 'class' in data.columns:
    data['class'] = data['class'].str.strip()

# 检查唯一值
print("目标列原始唯一值: ", data['class'].unique())
data = data[data['class'].isin(['ckd', 'notckd'])]
print("清理后目标列唯一值: ", data['class'].unique())

# LabelEncoder 编码目标列
le = LabelEncoder()
data['class'] = le.fit_transform(data['class'])
y = data['class'].values

# 查看编码后的类别映射
print("类别映射: ", dict(zip(le.classes_, le.transform(le.classes_))))

# 2. 缺失值处理和编码
numeric_cols = data.select_dtypes(include=['number']).columns
categorical_cols = data.select_dtypes(include=['object']).columns
```

目标列原始唯一值: ['ckd' 'notckd']
清理后目标列唯一值: ['ckd' 'notckd']
类别映射: {'ckd': 0, 'notckd': 1}
特征矩阵维度: (400, 25) 目标向量长度: 400
目标列类别分布: (array([0, 1]), array([250, 150]))
特征矩阵维度: (400, 25)
目标向量长度: 400

```
n_features = X.shape[1]
n_samples = X.shape[0]
y = 2 * y - 1 # 确保分类目标为 -1 和 1

# 动态分配成本
missing_ratios = np.array([0 if np.all(X[:, i] != '?') else (X[:, i] == '?').sum() / n_samples for i in
categorical_features = [2, 6, 7, 8, 9] # 类别特征
numerical_features = [i for i in range(n_features) if i not in categorical_features]

costs = np.zeros(n_features)
for i in range(n_features):
    costs[i] = 1.5 if i in categorical_features else 3.0
    costs[i] += 2.0 * missing_ratios[i]

# 筛选高相关性特征 得到X_high_corr
correlations = []
for i in range(X.shape[1]):
    if np.all(X[:, i] != '?'):
        corr = np.corrcoef(X[:, i], y)[0, 1]
    else:
        corr = 0
    correlations.append(corr)

high_corr_features = [i for i, corr in enumerate(correlations) if abs(corr) > 0.2]
if len(high_corr_features) == 0:
    high_corr_features = list(range(n_features))

X_high_corr = X[:, high_corr_features]
costs_high_corr = costs[high_corr_features]

# 缩放特征
scaler = MinMaxScaler(feature_range=(-0.01, 0.01))
X_high_corr = scaler.fit_transform(X_high_corr)
```


model building

```
l
model = Model("Cost_Effective_L1_SVM")
C1 = 8 # 分类误差权重
C2 = 0.0001 # 特征选择成本权重
CostLimit = sum(costs_high_corr) * 0.8
M = np.max(np.abs(X_high_corr)) * 10

# 添加变量
n_features_corr = X_high_corr.shape[1]
w_plus = model.addVars(n_features_corr, lb=0, name="w_plus")
w_minus = model.addVars(n_features_corr, lb=0, name="w_minus")
xi = model.addVars(n_samples, lb=0, name="xi")
s = model.addVars(n_features_corr, vtype=GRB.BINARY, name="s")
z = model.addVars(n_features_corr, lb=0, name="z")
```

```
# 目标函数
model.setObjective(
    quicksum(z[j] for j in range(n_features_corr)) +
    C2 * quicksum(s[j] * costs_high_corr[j] for j in range(n_features_corr)) +
    C1 * quicksum(xi[i] for i in range(n_samples)),
    GRB.MINIMIZE
)
r"""
```

object function

```
# 约束
for j in range(n_features_corr):
    model.addConstr(w_plus[j] - w_minus[j] <= z[j], f"z_plus_{j}")#L1 范数线性化
    model.addConstr(-(w_plus[j] - w_minus[j]) <= z[j], f"z_minus_{j}")#L1 范数线性化
    model.addConstr(w_plus[j] <= M * s[j], f"w_plus_s_{j}")#权重和特征选择的绑定
    model.addConstr(w_minus[j] <= M * s[j], f"w_minus_s_{j}")#权重和特征选择的绑定

model.addConstr(quicksum(s[j] * costs_high_corr[j] for j in range(n_features_corr)) <= CostLimit, "Cost_Li

for i in range(n_samples):#svm分类约束
    dot_product = quicksum((w_plus[j] - w_minus[j]) * X_high_corr[i, j] for j in range(n_features_corr))
    model.addConstr(y[i] * dot_product >= 1 - xi[i], f"classification_{i}")
```

constraints

```

# 数据准备
n_features = X.shape[1]
n_samples = X.shape[0]
y = 2 * y - 1 # 确保分类目标为 -1 和 1

# 动态分配成本
missing_ratios = np.array([0 if np.all(X[:, i] != '?') else (X[:, i] == '?').sum() / n_samples for i in range(n_features)])
categorical_features = [2, 6, 7, 8, 9] # 类别特征
numerical_features = [i for i in range(n_features) if i not in categorical_features]

costs = np.zeros(n_features)
for i in range(n_features):
    costs[i] = 1.5 if i in categorical_features else 3.0
    costs[i] += 2.0 * missing_ratios[i]

# 筛选高相关性特征 得到X_high_corr
correlations = []
for i in range(X.shape[1]):
    if np.all(X[:, i] != '?'):
        corr = np.corrcoef(X[:, i], y)[0, 1]
    else:
        corr = 0
    correlations.append(corr)

high_corr_features = [i for i, corr in enumerate(correlations) if abs(corr) > 0.2]
if len(high_corr_features) == 0:
    high_corr_features = list(range(n_features))

X_high_corr = X[:, high_corr_features]
costs_high_corr = costs[high_corr_features]

# 缩放特征
scaler = MinMaxScaler(feature_range=(-0.01, 0.01))
X_high_corr = scaler.fit_transform(X_high_corr)

#
# 模型搭建部分
model = Model("Cost_Effective_L1_SVM")
C1 = 8 # 分类误差权重
C2 = 0.0001 # 特征选择成本权重
CostLimit = sum(costs_high_corr) * 1
M = np.max(np.abs(X_high_corr)) * 10

```

```

# 添加变量
n_features_corr = X_high_corr.shape[1]
w_plus = model.addVars(n_features_corr, lb=0, name="w_plus")
w_minus = model.addVars(n_features_corr, lb=0, name="w_minus")
xi = model.addVars(n_samples, lb=0, name="xi")
s = model.addVars(n_features_corr, vtype=GRB.BINARY, name="s")
z = model.addVars(n_features_corr, lb=0, name="z")
# 目标函数
model.setObjective(
    quicksum(z[j] for j in range(n_features_corr)) +
    C2 * quicksum(s[j] * costs_high_corr[j] for j in range(n_features_corr)) +
    C1 * quicksum(xi[i] for i in range(n_samples)),
    GRB.MINIMIZE
)
# 约束
for j in range(n_features_corr):
    model.addConstr(w_plus[j] - w_minus[j] <= z[j], f"z_plus_{j}")#L1 范数线性化
    model.addConstr(-(w_plus[j] - w_minus[j]) <= z[j], f"z_minus_{j}")#L1 范数线性化
    model.addConstr(w_plus[j] <= M * s[j], f"w_plus_s_{j}")#权重和特征选择的绑定
    model.addConstr(w_minus[j] <= M * s[j], f"w_minus_s_{j}")#权重和特征选择的绑定
model.addConstr(quicksum(s[j] * costs_high_corr[j] for j in range(n_features_corr)) <= CostLimit, "Cost_Limit")#特征选择成本约束
for i in range(n_samples):#svm分类约束
    dot_product = quicksum((w_plus[j] - w_minus[j]) * X_high_corr[i, j] for j in range(n_features_corr))
    model.addConstr(y[i] * dot_product >= 1 - xi[i], f"classification_{i}")
# 优化模型
model.optimize()

# 提取结果
if model.status == GRB.OPTIMAL:
    selected_features = [j for j in range(n_features_corr) if s[j].X > 0.5]
    weights = np.array([w_plus[j].X - w_minus[j].X for j in selected_features])
    selected_features_cost = sum(costs_high_corr[j] for j in selected_features)

    best_result = {
        'C1': C1,
        'C2': C2,
        'CostLimit_factor': 0.7,
        'selected_features': selected_features,
        'weights': weights,
        'selected_features_cost': selected_features_cost,
        'total_cost': sum(costs_high_corr),
        'cost_limit': CostLimit,
        'optimization_status': "Optimal"
    }
}

```



```
# 动态分配成本 (考虑不确定性)
missing_ratios = np.array([0 if np.all(X[:, i] != '?') else (X[:, i] == '?').sum() / n_samples for i in range(n_features)])
categorical_features = [2, 6, 7, 8, 9] # 手动标注类别特征索引
numerical_features = [i for i in range(n_features) if i not in categorical_features]
```

```
costs = np.zeros(n_features)
for i in range(n_features):
    base_cost = 1.5 if i in categorical_features else 3.0
    missing_ratio = missing_ratios[i]
    costs[i] = base_cost + 2.0 * missing_ratio
```

```
# 定义成本的不确定性范围 (±10%)
cost_uncertainty_percentage = 0.1
costs_lower = costs * (1 - cost_uncertainty_percentage)
costs_upper = costs * (1 + cost_uncertainty_percentage)
```

```
# 筛选高相关性特征
correlations = []
for i in range(X.shape[1]):
    if np.all(X[:, i] != '?'):
        corr = np.corrcoef(X[:, i], y)[0, 1]
    else:
        corr = 0
    correlations.append(corr)
```

```
high_corr_features = [i for i, corr in enumerate(correlations) if abs(corr) > 0.2]
if len(high_corr_features) == 0:
    high_corr_features = list(range(n_features))
```

```
X_high_corr = X[:, high_corr_features]
costs_high_corr = costs[high_corr_features]
costs_upper_high_corr = costs_upper[high_corr_features]
```

```
# 缩放特征
scaler = MinMaxScaler(feature_range=(-0.01, 0.01))
X_high_corr = scaler.fit_transform(X_high_corr)
```

```
# Gurobi 模型
model = Model("Robust_Cost_Effective_L1_SVM")
C1 = 8 # 分类误差权重
C2 = 0.0001 # 特征选择成本权重
CostLimit = sum(costs_upper_high_corr) * 0.5 # 使用成本上界计算成本限制
M = np.max(np.abs(X_high_corr)) * 10
```

```
# Gurobi 模型
model = Model("Robust_Cost_Effective_L1_SVM")
C1 = 8 # 分类误差权重
C2 = 0.0001 # 特征选择成本权重
CostLimit = sum(costs_upper_high_corr) * 0.5 # 使用成本上界计算成本限制
M = np.max(np.abs(X_high_corr)) * 10
```

```
# 添加变量
n_features_corr = X_high_corr.shape[1]
w_plus = model.addVars(n_features_corr, lb=0, name="w_plus")
w_minus = model.addVars(n_features_corr, lb=0, name="w_minus")
xi = model.addVars(n_samples, lb=0, name="xi")
s = model.addVars(n_features_corr, vtype=GRB.BINARY, name="s")
z = model.addVars(n_features_corr, lb=0, name="z")
```

```
# 目标函数
model.setObjective(
    quicksum(z[j] for j in range(n_features_corr)) +
    C2 * quicksum(s[j] * costs_high_corr[j] for j in range(n_features_corr)) +
    C1 * quicksum(xi[i] for i in range(n_samples)),
    GRB.MINIMIZE
)
```

```
# 约束
for j in range(n_features_corr):
    model.addConstr(w_plus[j] - w_minus[j] <= z[j], f"z_plus_{j}")
    model.addConstr(-(w_plus[j] - w_minus[j]) <= z[j], f"z_minus_{j}")
    model.addConstr(w_plus[j] <= M * s[j], f"w_plus_s_{j}")
    model.addConstr(w_minus[j] <= M * s[j], f"w_minus_s_{j}")
```

```
# 修改后的成本约束 (使用成本上界)
model.addConstr(
    quicksum(s[j] * costs_upper_high_corr[j] for j in range(n_features_corr)) <= CostLimit,
    "Robust_Cost_Limit"
)
```

```
for i in range(n_samples):
    dot_product = quicksum((w_plus[j] - w_minus[j]) * X_high_corr[i, j] for j in range(n_features_corr))
    model.addConstr(y[i] * dot_product >= 1 - xi[i], f"classification_{i}")
```

```
# 优化模型
model.optimize()
```

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, accuracy_score, confusion_matrix
from sklearn.model_selection import train_test_split
import numpy as np
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42
)

# 初始化随机森林分类器
clf = RandomForestClassifier(n_estimators=100, random_state=42)

# 训练模型
clf.fit(X_train, y_train)
y_pred = clf.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)
report = classification_report(y_test, y_pred, target_names=['ckd', 'notckd'])

# 混淆矩阵
cm = confusion_matrix(y_test, y_pred)

print("模型准确率 (Accuracy): {:.4f}".format(accuracy))
print("\n分类报告 (Classification Report):\n", report)
print("混淆矩阵 (Confusion Matrix):\n", cm)
```

✓ 0.2s

Py

模型准确率 (Accuracy): 1.0000

分类报告 (Classification Report):

	precision	recall	f1-score	support
ckd	1.00	1.00	1.00	50
notckd	1.00	1.00	1.00	30
accuracy			1.00	80
macro avg	1.00	1.00	1.00	80
weighted avg	1.00	1.00	1.00	80

混淆矩阵 (Confusion Matrix):

```
[[50  0]
 [ 0 30]]
```


Presented by Group 21

Support Vector Machine for Cost-effective Feature Selection

WONG Man Chun, Matthew
TANG Cheuk Him, Thomas
XIONG Zelin
TIN Yiu Tsun, Jason
WEN Meng Qian, Velma

Solution
Recommendation
Conclusion

CESVM

True Label Distribution: Counter({-1: 250, 1: 150})
Predicted Label Distribution: Counter({-1: 358, 1: 42})

分类性能评估:

准确率 (Accuracy): 0.5800

	precision	recall	f1-score	support
Class -1	0.61	0.88	0.72	250
Class 1	0.29	0.08	0.12	150
accuracy			0.58	400
macro avg	0.45	0.48	0.42	400
weighted avg	0.49	0.58	0.50	400

成本效益分析:

总成本 (Selected Features Cost): 48.0

成本限制 (Cost Limit): 54.0

是否实现成本效益 (Cost-effective Selection): Yes

RCESVM

True Label Distribution: Counter({-1: 250, 1: 150})
Predicted Label Distribution: Counter({1: 242, -1: 158})

分类性能评估:

准确率 (Accuracy): 0.7700

	precision	recall	f1-score	support
Class -1	1.00	0.63	0.77	250
Class 1	0.62	1.00	0.77	150
accuracy			0.77	400
macro avg	0.81	0.82	0.77	400
weighted avg	0.86	0.77	0.77	400

成本效益分析 (考虑成本不确定性):

选定特征的总成本 (成本下界): 39.0

选定特征的总成本 (成本上界): 42.89999999999999

成本限制 (Cost Limit): 43.55999999999999

是否实现成本效益 (Cost-effective Selection): Yes

CESVM

```
True Label Distribution: Counter({-1: 250, 1: 150})
Predicted Label Distribution: Counter({-1: 260, 1: 140})

分类性能评估:
  准确率 (Accuracy): 0.5500
      precision    recall  f1-score   support

   Class -1      0.63     0.66     0.65     250
   Class 1       0.39     0.37     0.38     150

   accuracy          0.55     0.55     0.55     400
  macro avg       0.51     0.51     0.51     400
weighted avg       0.54     0.55     0.55     400

成本效益分析:
  总成本 (Selected Features Cost): 18.0
  成本限制 (Cost Limit): 18.6
  是否实现成本效益 (Cost-effective Selection): Yes
```

Cost limit factor = 0.4

```
True Label Distribution: Counter({-1: 250, 1: 150})
Predicted Label Distribution: Counter({-1: 260, 1: 140})

分类性能评估:
  准确率 (Accuracy): 0.5500
      precision    recall  f1-score   support

   Class -1      0.63     0.66     0.65     250
   Class 1       0.39     0.37     0.38     150

   accuracy          0.55     0.55     0.55     400
  macro avg       0.51     0.51     0.51     400
weighted avg       0.54     0.55     0.55     400

成本效益分析:
  总成本 (Selected Features Cost): 22.5
  成本限制 (Cost Limit): 23.25
  是否实现成本效益 (Cost-effective Selection): Yes
```

Cost limit factor = 0.5

```
True Label Distribution: Counter({-1: 250, 1: 150})
Predicted Label Distribution: Counter({-1: 297, 1: 103})

分类性能评估:
  准确率 (Accuracy): 0.5675
      precision    recall  f1-score   support

   Class -1      0.63     0.75     0.68     250
   Class 1       0.39     0.27     0.32     150

   accuracy          0.57     0.57     0.57     400
  macro avg       0.51     0.51     0.50     400
weighted avg       0.54     0.57     0.55     400

成本效益分析:
  总成本 (Selected Features Cost): 27.0
  成本限制 (Cost Limit): 27.9
  是否实现成本效益 (Cost-effective Selection): Yes
```

Cost limit factor = 0.6

```
True Label Distribution: Counter({-1: 250, 1: 150})
Predicted Label Distribution: Counter({-1: 333, 1: 67})

分类性能评估:
  准确率 (Accuracy): 0.5975
      precision    recall  f1-score   support

   Class -1      0.63     0.84     0.72     250
   Class 1       0.42     0.19     0.26     150

   accuracy          0.60     0.60     0.60     400
  macro avg       0.53     0.52     0.49     400
weighted avg       0.55     0.60     0.55     400

成本效益分析:
  总成本 (Selected Features Cost): 31.5
  成本限制 (Cost Limit): 32.55
  是否实现成本效益 (Cost-effective Selection): Yes
```

Cost limit factor = 0.7

RCESVM

```
True Label Distribution: Counter({-1: 250, 1: 150})
Predicted Label Distribution: Counter({1: 396, -1: 4})

分类性能评估:
 准确率 (Accuracy): 0.3850
      precision    recall  f1-score   support

   Class -1       1.00     0.02     0.03       250
   Class 1        0.38     1.00     0.55       150

 accuracy         0.39       400
 macro avg        0.69     0.51     0.29     400
weighted avg      0.77     0.39     0.23     400

成本效益分析 (考虑成本不确定性):
 选定特征的总成本 (成本下界): 18.0
 选定特征的总成本 (成本上界): 19.8
 成本限制 (Cost Limit): 20.459999999999994
 是否实现成本效益 (Cost-effective Selection): Yes
```

Cost limit factor = 0.4

```
True Label Distribution: Counter({-1: 250, 1: 150})
Predicted Label Distribution: Counter({1: 260, -1: 140})

分类性能评估:
 准确率 (Accuracy): 0.7250
      precision    recall  f1-score   support

   Class -1       1.00     0.56     0.72       250
   Class 1        0.58     1.00     0.73       150

 accuracy         0.72       400
 macro avg        0.79     0.78     0.72     400
weighted avg      0.84     0.72     0.72     400

成本效益分析 (考虑成本不确定性):
 选定特征的总成本 (成本下界): 22.5
 选定特征的总成本 (成本上界): 24.750000000000004
 成本限制 (Cost Limit): 25.574999999999992
 是否实现成本效益 (Cost-effective Selection): Yes
```

Cost limit factor = 0.5

```
True Label Distribution: Counter({-1: 250, 1: 150})
Predicted Label Distribution: Counter({1: 294, -1: 106})

分类性能评估:
 准确率 (Accuracy): 0.6400
      precision    recall  f1-score   support

   Class -1       1.00     0.42     0.60       250
   Class 1        0.51     1.00     0.68       150

 accuracy         0.64       400
 macro avg        0.76     0.71     0.64     400
weighted avg      0.82     0.64     0.63     400

成本效益分析 (考虑成本不确定性):
 选定特征的总成本 (成本下界): 27.0
 选定特征的总成本 (成本上界): 29.700000000000003
 成本限制 (Cost Limit): 30.689999999999999
 是否实现成本效益 (Cost-effective Selection): Yes
```

Cost limit factor = 0.6

```
True Label Distribution: Counter({-1: 250, 1: 150})
Predicted Label Distribution: Counter({1: 247, -1: 153})

分类性能评估:
 准确率 (Accuracy): 0.7575
      precision    recall  f1-score   support

   Class -1       1.00     0.61     0.76       250
   Class 1        0.61     1.00     0.76       150

 accuracy         0.76       400
 macro avg        0.80     0.81     0.76     400
weighted avg      0.85     0.76     0.76     400

成本效益分析 (考虑成本不确定性):
 选定特征的总成本 (成本下界): 31.5
 选定特征的总成本 (成本上界): 34.650000000000006
 成本限制 (Cost Limit): 35.804999999999986
 是否实现成本效益 (Cost-effective Selection): Yes
```

Cost limit factor = 0.7

RECOMMENDATION

HIGH ACCURACY MODEL



LESS FEATURES COULD BE SELECTED

***LESS COST AND TIME
CONSUMPTION***

MORE GRID FOR GRID SEARCH



MORE ACCURATE DATA

Feature Importances:

age: 0.2910

sodium: 0.1794

albumin: 0.1339

white blood cell count: 0.1208

sugar: 0.0737

diabetes mellitus: 0.0384

coronary artery disease: 0.0336

hypertension: 0.0313

packed cell volume: 0.0260

red blood cell count: 0.0206

hemoglobin: 0.0073

pedal edema: 0.0069

potassium: 0.0068

anemia: 0.0055

blood pressure: 0.0051

pus cell clumps: 0.0045

red blood cells: 0.0044

blood urea: 0.0036

serum creatinine: 0.0024

specific gravity: 0.0016

pus cell: 0.0014

blood glucose random: 0.0001

bacteria: 0.0000

appetite: 0.0000

CONCLUSION

Completing cost considerations into the feature selection process by using:

- L1-norm SVM
- CESVM
- RCESVM

We can derive feature sets that:

- Effective for prediction
- Feasible within resource constraints

➡ This framework supports better decision-making in real-world applications, ensuring that the selected features align with both performance goals and budgetary limitations.



THANKYOU

WONG Man Chun, Matthew
TANG Cheuk Him, Thomas
XIONG Zelin
TIN Yiu Tsun, Jason
WEN Meng Qian, Velma